

AIVOA.AI

API Schema & Backend Design

Round 2 connected QMS backend architecture and API design

Document 2 of 5

Professional presentation edition • Content preserved from the supplied source documents

5.1 Dashboard

5.2 Manufacturing / Batch

5.3 Deviation — retain and extend Round 1

5.4 Investigation

5.5 CAPA and Effectiveness

5.6 Connected context

6. Core Pydantic API Schemas

6.1 DeviationProcessResponse

```
{ "request_id": "uuid", "extraction": { "title": "Reactor temperature exceeded limit", "batch_number": "API-2026-041", "process_step": "Step 3 - Reaction", "parameter": "Temperature", "expected_range": "76-80 °C", "actual_value": "84 °C", "duration_minutes": 18, "description": "...", "confidence": 0.94 }, "assessment": { "recommended_severity": "major", "recommended_impact": "potential_product_quality_impact", "reason": "...", "confidence": 0.87 }, "evidence": [ { "document_id": "uuid", "document_name": "SOP-014 v3.2", "chunk_id": "uuid", "snippet": "...", "relevance": 0.91 } ], "requires_human_review": true }
```

6.2 Final DeviationCreateRequest

```
{ "batch_id": "uuid", "title": "Reactor temperature exceeded limit", "description": "...", "category": "process_parameter", "severity": "major", "reported_by_context": "server-derived user", "immediate_action": "...", "risk_assessment": { "impact": "potential_product_quality_impact", "rationale": "...", "criteria_version": "demo-qrm-v1" }, "ai_review": { "recommendation_id": "uuid", "decision": "accepted", "edited_fields": [] } }
```

6.3 Investigation schema

```
{ "id": "uuid", "investigation_number": "INV-2026-012", "deviation_id": "uuid", "status": "in_progress", "tasks": [ { "title": "Review batch manufacturing record", "owner_id": "uuid", "due_date": "2026-09-30", "status": "completed" } ], "root_cause": { "method": "5_whys", "problem": "Temperature reached 84 °C", "why_chain": [ "Cooling response was delayed", "Cooling valve did not respond correctly", "Valve actuator malfunctioned", "Maintenance failed to detect degradation" ], "final_root_cause": "Inadequate preventive-maintenance control for cooling-valve actuator", "contributing_factors": ["PM inspection frequency", "maintenance checklist"] }, "conclusion": "...", "completed_at": null }
```

6.4 CAPA schema

```
{ "id": "uuid", "capa_number": "CAPA-2026-009", "source_deviation_id": "uuid", "source_investigation_id": "uuid", "status": "in_progress", "root_cause": "...", "actions": [ { "type": "corrective", "title": "Replace faulty valve actuator", "owner_id": "uuid", "due_date": "2026-10-02", "status": "completed" }, { "type": "preventive", "title": "Update preventive maintenance procedure", "owner_id": "uuid", "due_date": "2026-10-05", "status": "in_progress" } ], "effectiveness": { "method": "monitor_next_5_batches", "criteria": "No recurrence", "status": "monitoring" } }
```

7. Workflow State Machine

The API should prevent arbitrary jumps in lifecycle state. The UI can show a stepper, but the backend remains the source of truth.

DEVIATION draft ↓ submit open ↓ start investigation under_investigation ↓ investigation complete root_cause_pending ↓ root cause approved capa_required ↓ CAPA created capa_in_progress ↓ actions complete effectiveness_pending ↓ effectiveness verified ready_for_closure ↓ authorized closure closed Allowed side paths: open → rejected/invalid (if triage rules require) capa_in_progress → overdue effectiveness_pending → ineffective → capa_in_progress closed → no normal edits; correction requires controlled change/audit path

7.1 Backend transition rules

A deviation cannot move to investigation unless the minimum deviation fields are present.

An investigation cannot be completed without a conclusion and required investigation tasks.

A root cause should be recorded before CAPA creation when the scenario requires CAPA.

CAPA closure should require all mandatory corrective/preventive actions to be complete and an effectiveness result where required by the demo workflow.

Closed records should be read-only through normal endpoints. Corrections should be represented by a controlled update event rather than silent mutation.

Every transition generates an audit event containing actor, old state, new state, reason, entity ID and request ID.

8. AI Integration Inside the Backend

The existing Round 1 AI pipeline should be retained and extended. The central rule is: AI produces evidence-backed recommendations; the human quality user owns the final record.

Upload / Text ↓ Document parsing ■■■■ Text PDF → direct extraction ■■■■ Scanned PDF → OCR fallback ↓ Structured extraction ↓ Pydantic validation ↓ Query construction ↓ Embedding ↓ pgvector retrieval ↓ Evidence package ↓ Deterministic checks ↓ LLM contextual recommendation ↓ Human review ↓ Persist final values ↓ Audit AI decision

8.1 AI responsibilities by Round 2 stage

8.2 AI recommendation audit record

```
{ "recommendation_id": "uuid", "entity_type": "deviation", "entity_id": "uuid", "stage": "triage", "model_provider": "groq",
  "model_name": "...", "model_version": "...", "prompt_version": "deviation-triage-v2", "input_hash": "...", "retrieved_sources":
  [ {"document_id": "uuid", "chunk_id": "uuid"} ], "recommendation": {...}, "confidence": 0.87, "human_decision": "accepted",
  "edited_fields": [], "created_at": "..." }
```

9. RAG and Evidence Design

Reference documents should be embedded once during ingestion. Runtime requests create a query embedding and retrieve a small set of relevant chunks. The retrieved evidence is passed to the model with clear source identifiers. RAG is a retrieval mechanism, not the final quality rule engine.

KnowledgeDocument ■■■■ SOP-014 v3.2 ■■■■ Batch procedure ■■■■ Investigation procedure ■■■■ CAPA procedure ↓ Chunk + metadata + embedding ↓ Runtime query ↓ Top-k retrieval ↓ Evidence context ↓ AI recommendation ↓ Source citations shown in UI

9.1 Evidence rules

Store document status/version and effective date in metadata.

Prefer approved/effective documents for operational recommendations.

Return document and chunk IDs with each recommendation.

If retrieval returns weak or no evidence, the AI should say that evidence is insufficient rather than inventing an answer.

Do not present generated content as a regulatory requirement unless the retrieved source actually supports it.

10. Security, Authorization and Data Integrity

Validate Supabase access token on every protected request.

Resolve user profile and active company/site membership server-side.

Never trust company_id, user_id or role from the request body for authorization.

Apply tenant filters to every repository query and use PostgreSQL RLS where configured.

Normal users cannot change their own role or membership.

Files should be stored in controlled storage and referenced by object ID rather than exposing raw storage paths as business identifiers.

AI-generated values must be marked as recommendations until human review is recorded.

Do not overwrite audit history. Append a new event.

Do not log passwords, access tokens, API keys or full sensitive document contents.

11. Error and Reliability Contract

```
{ "error_code": "AI_EXTRACTION_FAILED", "message": "The document could not be processed. No deviation was saved.", "request_id":
  "uuid", "details": { "stage": "structured_extraction", "retryable": true } }
```

12. Fast Round 2 Implementation Plan

12.1 What not to build for this round

Public registration or enterprise identity management.

Complex RBAC with dozens of permissions.

Full ERP, MES, inventory or procurement.

Full-scale complaint, recall or supplier portals.

Multiple-company switching.

A separate AI product unrelated to the deviation workflow.

A huge microservice architecture. One FastAPI service with clear modules is enough for the challenge.

13. Example: One Request From UI to Database

Example: the user sees an out-of-spec temperature during API manufacturing and clicks “Create Deviation”.

1. Browser POST /api/v1/deviations/process multipart/form-data: file = temperature_log.pdf context = batch API-2026-041 2. FastAPI Router - validate auth - create request_id - validate file type/size - call DeviationIntakeService 3. Service - extract text - OCR only if required - call AI extraction - validate structured JSON - build retrieval query 4. RAG - embed query - pgvector top-k search - return SOP/batch evidence 5. Assessment - deterministic expected-vs-actual check - LLM contextual explanation - return recommendation + evidence 6. Browser - show extracted fields - show severity recommendation - show evidence - require Accept/Edit/Reject 7. Final save POST /api/v1/deviations - server resolves actor/company - persist final deviation - persist AI review decision - append audit event - return DEV-2026-018 8. Next action POST /api/v1/deviations/{id}/start-investigation - create INV-2026-012 - transition deviation state - append audit event 9. Investigation - tasks/evidence/root cause - complete investigation 10. CAPA - create CAPA from root cause - execute actions - record effectiveness 11. Closure - authorized user closes CAPA/deviation - dashboard counters update

14. Example API Calls

14.1 Start investigation

POST /api/v1/deviations/3f2.../start-investigation Authorization: Bearer <supabase-access-token> Content-Type: application/json { "owner_id": "qa-user-uuid", "reason": "Deviation requires documented investigation." } Response 201: { "investigation_id": "8ab...", "investigation_number": "INV-2026-012", "deviation_id": "3f2...", "status": "in_progress", "deviation_status": "under_investigation" }

14.2 Create CAPA from root cause

POST /api/v1/capas { "deviation_id": "3f2...", "investigation_id": "8ab...", "root_cause": "Inadequate preventive-maintenance control...", "actions": [{ "type": "corrective", "title": "Replace faulty valve actuator", "owner_id": "eng-user-uuid", "due_date": "2026-10-02" }, { "type": "preventive", "title": "Update preventive maintenance procedure", "owner_id": "eng-user-uuid", "due_date": "2026-10-05" }] }

15. Recommended Round 2 Backend Folder Structure

backend/app/ ■■■■ api/v1/ ■■■■ router.py ■■■■ dashboard.py ■■■■ batches.py ■■■■ deviations.py ■■■■ investigations.py ■■■■ capas.py ■■■■ batch_release.py ■■■■ complaints.py ■■■■ suppliers.py ■■■■ ai/ ■■■■ graph.py ■■■■ extraction.py ■■■■ investigation.py ■■■■ capa.py ■■■■ risk_assessment.py ■■■■ prompts.py ■■■■ models.py ■■■■ models/ ■■■■ batch.py ■■■■ process_check.py ■■■■ deviation.py ■■■■ investigation.py ■■■■ capa.py ■■■■ complaint.py ■■■■ supplier.py ■■■■ audit.py ■■■■ ai_interaction.py ■■■■ schemas/ ■■■■ repositories/ ■■■■ services/ ■■■■ batch_service.py ■■■■ deviation_service.py ■■■■ investigation_service.py ■■■■ capa_service.py ■■■■ complaint_service.py ■■■■ supplier_service.py ■■■■ rag/ ■■■■ core/

16. Backend Acceptance Checklist

17. Key Design Decision

The Round 2 backend should feel like one connected quality record, not a collection of CRUD screens. The central object is the quality event/deviation. It carries the API batch context, process evidence, investigation, root cause, CAPA, effectiveness result and closure history. Batch release, complaints and supplier quality are connected views around that same traceability graph. This directly supports the challenge's required understanding of how QMS processes connect while keeping implementation realistic for the available time.

END OF FILE 1

Document	Scope	Priority
API & Backend Design	FastAPI contracts, services, workflow state, AI orchestration, persistence boundaries, security and error handling	P0
Round 2 core lifecycle	Deviation → Investigation → Root Cause → CAPA → Effectiveness → Closure	P0
Connected context	Batch, in-process checks, batch release, complaints, supplier quality	P1 / light implementation

Existing capability	Current implementation	Round 2 action
API framework	FastAPI	Keep; add routers/services
Database	SQLAlchemy + PostgreSQL/Supabase	Keep; add QMS tables and relations

Existing capability	Current implementation	Round 2 action
Authentication	Supabase Auth + backend identity validation	Keep
Tenant isolation	Company/site membership + authorization checks	Keep and extend to new records
Deviation APIs	Process, create, list, get, update	Keep; add workflow transitions and links
AI extraction	Structured Pydantic extraction	Reuse for deviation/investigation evidence
RAG	pgvector + knowledge chunks	Reuse for SOP/procedure evidence
Risk	Deterministic checks + LLM explanation	Reuse; expose as a formal assessment object
Audit	Audit events	Extend to all lifecycle transitions and AI decisions
Frontend state	React + Redux Toolkit	Add slices for batch, investigation, CAPA, closure

Layer	Responsibility	Must not do
Router	HTTP contract, auth dependency, request/response serialization	Business logic or direct SQL
Service	Workflow orchestration, state transitions, business rules	Render UI or expose DB internals
Repository	CRUD/query operations and transaction boundaries	Decide workflow state
AI orchestration	Extraction, retrieval, recommendation, evidence packaging	Persist final GMP decision without review
Models	Relational persistence and constraints	Contain UI-specific behavior
Schemas	Pydantic request/response validation	Trust frontend identity/tenant fields
Audit service	Immutable lifecycle/decision events	Allow arbitrary client-supplied audit entries

Convention	Decision
IDs	UUID internally; human-readable record numbers such as DEV-2026-018 for UI
Dates	ISO-8601 UTC timestamps
Pagination	limit + cursor or page/limit; use cursor for large activity feeds
Errors	Consistent {error_code, message, request_id, details?}
Idempotency	Support Idempotency-Key on create/transition endpoints where duplicate clicks could create duplicate records
Optimistic concurrency	Use version or updated_at checks on critical workflow updates
Audit	Server-generated audit event on create/update/transition/AI review
Tenant filtering	Applied in service/repository and reinforced by database RLS where available

Method	Endpoint	Purpose
GET	/dashboard/summary	Counts for deviations, investigations, CAPAs, pending batch releases and complaints
GET	/dashboard/activity	Recent quality events across linked modules
GET	/dashboard/trends	Time-series data for deviations/CAPA/complaints

Method	Endpoint	Purpose
POST	/batches	Create a demo/API manufacturing batch
GET	/batches	List authorized batches
GET	/batches/{batch_id}	Get batch with raw materials, process steps and linked quality records
POST	/batches/{batch_id}/process-checks	Record an in-process check
GET	/batches/{batch_id}/process-checks	List process checks
POST	/batches/{batch_id}/release-review	Create or update batch release review

Method	Endpoint	Purpose
POST	/deviations/process	Upload/text intake → extraction + AI assessment preview
POST	/deviations	Create final human-reviewed deviation
GET	/deviations	List authorized deviations with filters
GET	/deviations/{id}	Get deviation details and linked records
PUT	/deviations/{id}	Update editable fields while state permits

Method	Endpoint	Purpose
POST	/deviations/{id}/assign	Assign investigator/owner
POST	/deviations/{id}/start-investigation	Create investigation and move lifecycle state
POST	/deviations/{id}/risk-assessment	Save/recalculate risk assessment
GET	/deviations/{id}/links	Return linked batch, checks, investigation, CAPA, complaint, supplier events

Method	Endpoint	Purpose
POST	/investigations	Create investigation from deviation
GET	/investigations	List investigations
GET	/investigations/{id}	Get investigation with plan, evidence, analysis and root cause
POST	/investigations/{id}/tasks	Create investigation task
PATCH	/investigations/{id}/tasks/{task_id}	Update task status/owner/due date
POST	/investigations/{id}/evidence	Attach evidence metadata/file reference
POST	/investigations/{id}/root-cause	Save root-cause analysis
POST	/investigations/{id}/conclusion	Record investigation conclusion
POST	/investigations/{id}/complete	Complete investigation after required fields are present

Method	Endpoint	Purpose
POST	/capas	Create CAPA from a deviation/investigation
GET	/capas	List CAPAs with status filters
GET	/capas/{id}	Get CAPA and linked source records
POST	/capas/{id}/actions	Add corrective or preventive action
PATCH	/capas/{id}/actions/{action_id}	Update action owner, due date, status and evidence
POST	/capas/{id}/effectiveness	Create effectiveness plan/check
POST	/capas/{id}/effectiveness/result	Record effectiveness evidence/result
POST	/capas/{id}/close	Close CAPA only after required effectiveness/approval rules pass

Area	Endpoints
Complaints	POST /complaints, GET /complaints, GET /complaints/{id}, POST /complaints/{id}/investigation
Supplier Quality	GET /suppliers, GET /suppliers/{id}, POST /suppliers/{id}/quality-events, POST /suppliers/{id}/capa
Batch Release	GET /batch-releases/{id}, POST /batch-releases/{id}/decision
Adverse Events	POST /adverse-events, GET /adverse-events/{id} — lightweight context only for Round 2

Stage	AI can do	Human must do
Deviation intake	Extract batch, process step, parameter, expected/actual value, dates and narrative	Confirm extracted facts
Triage	Suggest category/severity/impact and explain why	Accept, edit or reject recommendation
Investigation	Retrieve relevant SOPs, batch records and prior similar events; suggest investigation questions	Choose investigation scope and evidence
Root cause	Suggest candidate causes and a 5-Whys draft grounded in evidence	Confirm the actual root cause
CAPA	Draft corrective/preventive actions linked to root cause	Approve actions, owner and due date
Effectiveness	Summarize monitoring evidence and flag recurrence	Decide whether CAPA is effective
Closure	Prepare closure summary	Authorize final closure

Failure	Backend behavior
Invalid/unsupported PDF	Return 400 with friendly validation message; do not create a final deviation
OCR failure	Return controlled error; permit retry; preserve upload reference if safe

Failure	Backend behavior
LLM timeout/rate limit	Retry with bounded backoff; if still failing, return processing error and do not fabricate results
Malformed LLM JSON	Validate with Pydantic; attempt constrained repair once or reject safely
RAG unavailable	Do not claim evidence-backed recommendation; return degraded result requiring human review
Database failure	Transaction rollback; never return success if final persistence failed
Duplicate create	Use Idempotency-Key where implemented; return existing record for same request key
Unauthorized	401 for missing/invalid auth; 403 for authenticated but unauthorized
Stale update	409 conflict if version/updated_at no longer matches

Phase	Build	Outcome
P0	WorkflowStepper + Deviation state transitions	Existing deviation visibly becomes a lifecycle
P0	Investigation APIs + UI	Deviation can start and complete an investigation
P0	Root cause / 5 Whys	Investigation produces an explicit root cause
P0	CAPA APIs + UI	Root cause produces corrective/preventive actions
P0	Effectiveness + closure	CAPA can be verified and closed
P1	Batch + in-process check context	Deviation is grounded in API manufacturing
P1	Batch release	Quality record affects disposition context
P1	Complaint + supplier lightweight records	Shows downstream/ upstream QMS connections
P1	Dashboard aggregation	Evaluator can see complete QMS picture

Requirement	Done when...
API manufacturing context	A batch has raw material, manufacturing and in-process check records
Deviation	Out-of-spec check can create a deviation with linked batch/check
AI assist	AI extraction returns structured fields + evidence + recommendation
Human review	Final save requires explicit human decision on AI recommendations
Investigation	Deviation can create investigation with tasks/evidence
Root cause	5 Whys/root-cause result is stored and linked to investigation
CAPA	Corrective and preventive actions are linked to root cause
Effectiveness	Effectiveness criteria/results are stored
Closure	Authorized user can close after required conditions
Batch release	Release review can see deviation/CAPA status
Complaint	Complaint can reference product/batch/deviation
Supplier	Supplier quality event can reference raw material/deviation
Auditability	Lifecycle and AI review events are recorded
Security	All records are tenant-scoped and role checked
Reliability	AI/DB failures never produce false success